Software Technology I

Week 6 Tutorial

Arnita Ahmed – ID: u3313373 – Timing: Wednesday, 9.30 AM

Case Study 3: Keras Package and Google Collab

<u>Task 1.1</u>

Code

Double-click (or enter) to edit

[5]

```
import matplotlib
import keras
print(matplotlib.__version__)
print(keras.__version__)

import matplotlib.pyplot as plt
from keras.preprocessing.image import load_img

img_path = '/content/1_Gammar 2021_1_2021_06_03-13-19-23-856.PNG'
img = load_img(img_path)

print(type(img))
print(img.format)
print(img.mode)
print(img.size)

plt.imshow(img)
plt.axis('off')
plt.show()
```

Output

```
3.10.0
3.13.2
<class 'PIL.PngImagePlugin.PngImageFile'>
PNG
RGB
(464, 464)
```

Task Explanation = This script loads an insect image using the load_img() function from Keras. It then prints the image type, format, color mode and size so the image properties can be checked. After that, Matplotlib is used to display the image on the screen, and the axis is turned off to make the output cleaner.

<u>Task 1.2</u>

Code

Double-click (or enter) to edit

[7]
0s

```
from keras.preprocessing.image import load_img
from keras.preprocessing.image import img_to_array
from keras.preprocessing.image import array_to_img

img_path = '/content/1_Gammar 2021_1_2021_06_03-13-19-23-856.PNG'
img = load_img(img_path)

print(type(img))

img_array = img_to_array(img)
print(img_array.dtype)
print(img_array.shape)

img_pil = array_to_img(img_array)
print(type(img_pil))
```

Output

```
<class 'PIL.PngImagePlugin.PngImageFile'>
float32
(464, 464, 3)
<class 'PIL.Image.Image'>
```

Task Explanation = This script loads an image using Keras and converts it into a NumPy array using img_to_array(). This is done because machine learning models usually work with numerical arrays rather than image files. The script then prints the data type and shape of the array and converts the array back into an image using array_to_img().

TASK 1.3

Code

```python
import matplotlib.pyplot as plt
import os
from keras.preprocessing.image import load_img
from keras.preprocessing.image import save_img
from keras.preprocessing.image import img_to_array

img_path = '/content/1_Gammar 2021_1_2021_06_03-13-19-23-856.PNG'

# load image in grayscale
img = load_img(img_path, color_mode='grayscale')

# convert image to array
img_array = img_to_array(img)

# create save path
save_path_dir = '/content/'
save_filename = 'gammar_grayscale.jpg'
full_save_path = os.path.join(save_path_dir, save_filename)
```

```python
# create save path
save_path_dir = '/content/'
save_filename = 'gammar_grayscale.jpg'
full_save_path = os.path.join(save_path_dir, save_filename)

# save grayscale image
save_img(full_save_path, img_array)

# reload saved image
img = load_img(full_save_path)

# print image details
print(type(img))
print(img.format)
print(img.mode)
print(img.size)

# display image
plt.imshow(img)
plt.axis('off')
plt.show()
```

Output

```
<class 'PIL.Image.Image'>
None
RGB
(464, 464)
```

Task Explanation = The script loads an image in grayscale mode, which means the image is converted from colour into shades of gray. The grayscale image is then turned into a NumPy array and saved as a new file using save_img(). After saving, the new image is loaded again and displayed with Matplotlib to confirm that it was saved correctly.

<u>TASK 1.4</u>

Code

```
[10]
3s
# load and summarize the MNIST dataset
from keras.datasets import mnist
# load dataset
(train_images, train_labels), (test_images,
test_labels) = mnist.load_data()
# summarize dataset shape
print('Train', train_images.shape,
train_labels.shape)
print('Test', (test_images.shape,
test_labels.shape))
# summarize pixel values
print('Train', train_images.min(),
train_images.max(), train_images.mean(),
train_images.std())
print('Train', test_images.min(),
test_images.max(), test_images.mean(),
test_images.std())
```

Output

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
Train (60000, 28, 28) (60000,)
Test ((10000, 28, 28), (10000,))
Train 0 255 33.318421449829934 78.56748998339798
Train 0 255 33.791224489795916 79.17246322228644
```

Task Explanation = This script loads the MNIST dataset from Keras. MNIST is a dataset of handwritten digit images used for machine learning. The code prints the shape of the training and testing datasets to show how many images and labels are included. It also prints the minimum, maximum, mean, and standard deviation of the pixel values to summarize the dataset.

TASK 1.5

Code

```python
# example of normalizing an image dataset (MNIST)
from keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
# load dataset
(trainX, trainY), (testX, testY) = mnist.load_data()
# reshape dataset to have a single channel
width, height, channels = trainX.shape[1], trainX.shape[2], 1
trainX = trainX.reshape((trainX.shape[0], width, height, channels))
testX = testX.reshape((testX.shape[0], width, height, channels))
# confirm scale of pixels
print('Train min=%.3f, max=%.3f' % (trainX.min(), trainX.max()))
print('Test min=%.3f, max=%.3f' % (testX.min(), testX.max()))
# create generator (1.0/255.0 = 0.003921568627451)
datagen = ImageDataGenerator(rescale=1.0/255.0)
# Note: there is no need to fit the generator in this case
# prepare a iterators to scale images
train_iterator = datagen.flow(trainX, trainY, batch_size=64)
test_iterator = datagen.flow(testX, testY, batch_size=64)
print('Batches train=%d, test=%d' % (len(train_iterator),
len(test_iterator)))
# confirm the scaling works
batchX, batchy = next(train_iterator)
print('Batch shape=%s, min=%.3f, max=%.3f' % (batchX.shape,
batchX.min(), batchX.max()))
```

Output

```
Train min=0.000, max=255.000
Test min=0.000, max=255.000
Batches train=938, test=157
Batch shape=(64, 28, 28, 1), min=0.000, max=1.000
```

Task Explanation = This script loads the MNIST dataset and reshapes the images so they include one color channel. It uses ImageDataGenerator(rescale=1.0/255.0) to normalize the pixel values from the range 0 to 255 into the range 0 to 1. This makes the data easier for machine learning models to process. The script checks the new values by printing the batch shape and the new minimum and maximum pixel values.

TASK 1.6

Code

```
[ ]
# example of centering a image dataset
from keras.datasets import mnist
from tensorflow.keras.preprocessing.image import
ImageDataGenerator
# load dataset
(trainX, trainy), (testX, testy) = mnist.load_data()
# reshape dataset to have a single channel
width, height, channels = trainX.shape[1], trainX.shape[2], 1
trainX = trainX.reshape((trainX.shape[0], width, height,
channels))
testX = testX.reshape((testX.shape[0], width, height,
channels))
# report per-image mean
print('Means train=%.3f, test=%.3f' % (trainX.mean(),
testX.mean()))
# create generator that centers pixel values
datagen = ImageDataGenerator(featurewise_center=True)
# calculate the mean on the training dataset
datagen.fit(trainX)
print('Data Generator Mean: %.3f' % datagen.mean.item())
# demonstrate effect on a single batch of samples
iterator = datagen.flow(trainX, trainy, batch_size=64)
```

```
# demonstrate effect on a single batch of samples
iterator = datagen.flow(trainX, trainy, batch_size=64)
# get a batch
batchX, batchy = next(iterator)
# mean pixel value in the batch
print(batchX.shape, batchX.mean())
# demonstrate effect on entire training dataset
iterator = datagen.flow(trainX, trainy,
batch_size=len(trainX), shuffle=False)
# get a batch
batchX, batchy = next(iterator)
# mean pixel value in the batch
print(batchX.shape, batchX.mean())
```

Output

```
Means train=33.318, test=33.791
Data Generator Mean: 33.318
(64, 28, 28, 1) 0.6017728
(60000, 28, 28, 1) -1.9512918e-05
```

Task Explanation = This script centers the MNIST dataset by subtracting the overall mean pixel value from each image. First, it prints the average pixel value of the training and testing data. Then ImageDataGenerator(featurewise_center=True) is used to calculate the dataset mean and center the images. The output shows that the new batch mean moves closer to zero after centering.

<u>TASK 1.7</u>

Code

```
[14]
✓ 2s
# example of standardizing a image dataset
from keras.datasets import mnist
from tensorflow.keras.preprocessing.image import ImageDataGenerator
# load dataset
(trainX, trainy), (testX, testy) = mnist.load_data()
# reshape dataset to have a single channel
width, height, channels = trainX.shape[1], trainX.shape[2], 1
trainX = trainX.reshape((trainX.shape[0], width, height, channels))
testX = testX.reshape((testX.shape[0], width, height, channels))
# report pixel means and standard deviations
print('Statistics train=%.3f (%.3f), test=%.3f (%.3f)' % (trainX.mean(), trainX.std(),
testX.mean(), testX.std()))
# create generator that centers pixel values
datagen = ImageDataGenerator(featurewise_center=True,
featurewise_std_normalization=True)
# calculate the mean on the training dataset
datagen.fit(trainX)
print('Data Generator mean=%.3f, std=%.3f' % (datagen.mean.item(),
datagen.std.item()))
```

```
print('Data Generator mean=%.3f, std=%.3f' % (datagen.mean.item(),
datagen.std.item()))
# demonstrate effect on a single batch of samples
iterator = datagen.flow(trainX, trainy, batch_size=64)
# get a batch
batchX, batchy = next(iterator)
# pixel stats in the batch
print(batchX.shape, batchX.mean(), batchX.std())
# demonstrate effect on entire training dataset
iterator = datagen.flow(trainX, trainy, batch_size=len(trainX), shuffle=False)
# get a batch
batchX, batchy = next(iterator)
# pixel stats in the batch
print(batchX.shape, batchX.mean(), batchX.std())
```

Output

```
•••  Statistics train=33.318 (78.567), test=33.791 (79.172)
     Data Generator mean=33.318, std=78.567
     (64, 28, 28, 1) 0.005947325 1.0069932
     (60000, 28, 28, 1) -3.4560264e-07 0.9999998
```

Task Explanation =
This script standardizes the MNIST dataset. Standardization means the data is first centered by subtracting the mean, and then scaled by dividing by the standard deviation. This is done using featurewise_center=True and featurewise_std_normalization=True in ImageDataGenerator. The script prints the mean and standard deviation before and after transformation to show the effect.